

本节例题列表

本节讲解的例题及其囊括的知识点，如表4-1所示。

表4-1 基础数据结构例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 4-1	POJ 2051	Argus	优先级队列	陈锋
例 4-2	POJ 1363	Rails	栈的使用	陈锋
例 4-3	UVa 210	Concurrency Simulator	双端队列	陈锋
例 4-4	UVa 548	Tree	从中序和后序恢复二叉树	陈锋
例 4-5	UVa 699	The Falling Leaves	二叉树的 DFS	陈锋
例 4-6	UVa 839	Not so Mobile	二叉树隐式 DFS	陈锋
例 4-7	UVa 297	Quadrees	四分树	陈锋
例 4-8	UVa 442	Matrix Chain Multiplication	使用栈进行表达式计算	刘汝佳
例 4-9	UVa 10410	Tree Reconstruction	树的 BFS 与 DFS 序列转换	陈锋
例 4-10	UVa 11997	K Smallest Sums	优先级队列与 K 路合并	刘汝佳
例 4-11	UVa 122	Trees on the level	树的层次遍历	陈锋
例 4-12	UVa 12657	Boxes in a Line	双向链表	刘汝佳
例 4-13	UVa 1662	Brackets Removal	递归下降法：表达式树	陈锋
例 4-14	UVa 1160	X-Plosives	并查集	陈锋
例 4-15	POJ 1962	Corporative Network	带权并查集	刘汝佳
例 4-16	UVa 572	Oil Deposits	图的连通块	陈锋

4.2 区间信息维护

本节主要介绍基于区间信息维护数据结构（包括树状数组、RMQ、线段树）的经典例题以及代码实现。

例4-17 【树状数组；维护逆序对】乒乓球比赛（Ping pong，北京2008，POJ 3928）

一条大街上住着 n ($3 \leq n \leq 20\,000$) 个乒乓球爱好者，他们经常组织比赛切磋技术。每个人都有一个不同的技能值 a_i ($1 \leq a_i \leq 100\,000$)。每场比赛需要3个人：两名选手，一名裁判。他们有一个奇怪

的规定，即裁判必须住在两名选手的中间，并且技能值也在两名选手之间。问一共能组织多少场比赛。

【代码实现】

```
// 陈锋
#include <cassert>
#include <iostream>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;

template <typename T, size_t SZ>
struct BIT { // 树状数组（二叉索引树）
    T C[SZ];
    size_t N;
    inline void init(size_t sz) {
        N = sz;
```

```

    assert(N + 1 < SZ);
    fill_n(C, N + 1, 0);
}
inline int lowbit(int x) { return x & -x; }
inline T sum(size_t i) { //  $\sum(k = 1 \rightarrow i)$ 
    T ans = 0;
    while (i > 0) ans += C[i], i -= lowbit(i);
    return ans;
}

inline void add(size_t i, const T& v) {
    while (i <= N) C[i] += v, i += lowbit(i);
}
};

const int MAXN = 20000 + 4, MAXA = 1e5;
int A[MAXN], C[MAXN], D[MAXN];
/*
    i 当裁判, 考虑 a[1~i-1] 中有 ci 个比 ai 小, (i-1)-ci 个比 ai 大,
    a[i+1~n] 有 di 个比 ai 小, (n-i-di) 个比 ai 大,
    则 i 当裁判就有 ci(n-i-di) + (i-1-ci)di 场比赛, 求  $\Sigma$  即可。
    ci, di 扫描求得, X[a] = 1 -> exist Ai = a before
*/
int main() {
    int T, N;
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> T;
    BIT < int, MAXA + 4 > X;
    while (T--) {
        cin >> N, fill_n(C, N + 1, 0);
        X.init(MAXA);
        _rep(i, 1, N) cin >> A[i], C[i] = X.sum(A[i] - 1), X.add(A[i], 1);
        X.init(MAXA);
        LL ans = 0;
        for (int i = N; i >= 1; i--) {
            int d = X.sum(A[i] - 1);
            X.add(A[i], 1);
            if (i < N && i > 1) ans += C[i] * (N - i - d) + (i - 1 - C[i]) * d;
        }
        cout << ans << endl;
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.2.2 节例题 7
注意: 本题将常规的树状数组泛型化, 实现了其内部数据类型以及数组大小的参数化
同类题目: UVa 11525, Permutation
*/

```

例 4-18 【RMQ】 频繁出现的数值 (Frequent Values, POJ 3368)

给出一个非降序排列的整数数组 $a_1, a_2, \dots, a_n$ ($1 \leq n, q \leq 100\,000$, $-100\,000 \leq a_i \leq 100\,000$)，你的任务是对于一系列询问 (i, j) ，分析 $a_i, a_{i+1}, \dots, a_j$ 中出现次数最多的值，给出具体出现的次数。

【输出格式】

对于每个询问，输出结果。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <vector>
using namespace std;

const int NN = 1e5 + 8, maxlog = 20;

// 区间最大值
struct RMQ {
    int d[NN][maxlog];
    void init(const vector<int>& A) {
        int n = A.size();
        for (int i = 0; i < n; i++) d[i][0] = A[i];
        for (int j = 1; (1 << j) <= n; j++)
            for (int i = 0; i + (1 << j) - 1 < n; i++)
                d[i][j] = max(d[i][j - 1], d[i + (1 << (j - 1))][j - 1]);
    }

    int query(int L, int R) {
        int k = 0;
        while ((1 << (k + 1)) <= R - L + 1)
            k++;
        return max(d[L][k], d[R - (1 << k) + 1][k]);
    }
};

int a[NN], num[NN], left[NN], right[NN];
RMQ rmq;

int main() {
    for (int n, q; scanf("%d%d", &n, &q) == 2;) {
        for (int i = 0; i < n; i++) scanf("%d", &a[i]);
        a[n] = a[n - 1] + 1; // 哨兵
        vector<int> count;
        for (int i = 0, start = -1; i <= n; i++) {
            if (i == 0 || a[i] > a[i - 1]) { // 新段开始
                if (i > 0) {
                    count.push_back(i - start);
                    for (int j = start; j < i; j++) {
                        num[j] = count.size() - 1;
                        left[j] = start, right[j] = i - 1;
                    }
                }
                start = i;
            }
        }
        rmq.init(count);
        for (int L, R, ans; q--;) {
            scanf("%d%d", &L, &R), L--, R--;
            if (num[L] == num[R])
                ans = R - L + 1;
            else {
                ans = max(R - left[R] + 1, right[L] - L + 1);
                if (num[L] + 1 < num[R])
                    ans = max(ans, rmq.query(num[L] + 1, num[R] - 1));
            }
            printf("%d\n", ans);
        }
        return 0;
    }
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.2.2 节例题 8
注意：本题通过增加哨兵元素，简化了数组边界的处理
同类题目：Interviewe, HDU 3486
*/

```

例4-19 【线段树；维护区间前后缀】动态最大连续和（Ray, Pass me the Dishes, UVa 1400）

给出一个长度为 n 的整数序列 D ，你的任务是对 m ($1 \leq n, m \leq 500\,000$) 个询问做出回答。对于询问 (a, b) ，需要找到两个下标 x 和 y ，使得 $a \leq x \leq y \leq b$ ，并且 $D_x + D_{x+1} + \dots + D_y$ 尽量大。如果有多组满足条件的 x 和 y ， x 应该尽量小。如果还有多解， y 应该尽量小。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
typedef pair<int, int> Interval;
const int MAXN = 5e5 + 4;
LL SD[MAXN];
inline LL sum(int L, int R) {                // [L,R]
    // assert(L <= R);
    return SD[R] - SD[L - 1];
}
```

```

inline LL sum(const Interval& i) { return sum(i.first, i.second); }
inline Interval maxI(const Interval& i1, const Interval& i2) {
    LL s1 = sum(i1), s2 = sum(i2);
    if (s1 != s2) return s1 > s2 ? i1 : i2;
    return min(i1, i2);
}

struct MaxVal {
    int pfx, sfx;
    Interval sub;
};

struct IntervalTree {
    MaxVal Nodes[MAXN * 2];
    int qL, qR, N;
    void build(int N) { // [1, N]
        this->N = N;
        build(1, N, 1);
    }

    void build(int L, int R, int O) {
        assert(L <= R);
        assert(O > 0);
        if (L == R) {
            Nodes[O] = {L, L, make_pair(L, L)};
            return;
        }
        int M = (L + R) / 2, lc = 2 * O, rc = 2 * O + 1;
        build(L, M, lc), build(M + 1, R, rc);
        const MaxVal &nl = Nodes[lc], &nr = Nodes[rc];
        MaxVal &no = Nodes[O];
        no.pfx = sum(L, nl.pfx) >= sum(L, nr.pfx) ? nl.pfx : nr.pfx;
        no.sfx = sum(nl.sfx, R) >= sum(nr.sfx, R) ? nl.sfx : nr.sfx;
        no.sub = maxI(nl.sub, nr.sub);
        no.sub = maxI(no.sub, make_pair(nl.sfx, nr.pfx));
    }

    Interval query(int l, int r) { // [1, r]中的最大子区间[a, b]
        assert(l <= r);
        qL = l, qR = r;
        return _query(1, N, 1);
    }

    Interval _query(const int L, const int R, const int O) {
        if (qL <= L && R <= qR) return Nodes[O].sub;
        int M = (L + R) / 2, lc = O * 2, rc = 2 * O + 1;
        if (qR <= M) return _query(L, M, lc);
        if (qL > M) return _query(M + 1, R, rc);
        Interval ans = make_pair(_querySfx(L, M, lc), _queryPfx(M + 1, R, rc));
        ans = maxI(ans, maxI(_query(L, M, lc), _query(M + 1, R, rc)));
    }
};

```

```

        return ans;
    }

    int _queryPfx(const int L, const int R, const int O) {
        if (qL <= L && R <= qR) return Nodes[O].pfx;
        int M = (L + R) / 2, lc = 2 * O, rc = 2 * O + 1;
        if (qR <= M) return _queryPfx(L, M, lc);
        if (qL > M) return _queryPfx(M + 1, R, rc);
        int m1 = _queryPfx(L, M, lc), m2 = _queryPfx(M + 1, R, rc);
        return sum(L, m1) >= sum(L, m2) ? m1 : m2;
    }

    int _querySfx(const int L, const int R, const int O) {
        if (qL <= L && R <= qR) return Nodes[O].sfx;
        int M = (L + R) / 2, lc = O * 2, rc = 2 * O + 1;
        if (qR <= M) return _querySfx(L, M, lc);
        if (qL > M) return _querySfx(M + 1, R, rc);
        int m1 = _querySfx(L, M, lc), m2 = _querySfx(M + 1, R, rc);
        return sum(m1, R) >= sum(m2, R) ? m1 : m2;
    }
};

IntervalTree tree;

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    SD[0] = 0;
    for (int t = 1, d, a, b, N, M; cin >> N >> M; t++) {
        _rep(i, 1, N) cin >> d, SD[i] = SD[i - 1] + d;
        tree.build(N);
        printf("Case %d:\n", t);
        _rep(i, 1, M) {
            cin >> a >> b;
            Interval ans = tree.query(a, b);
            printf("%d %d\n", ans.first, ans.second);
        }
    }
    return 0;
}

/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.2.4 节例题 9
注意: 本题通过 Interval 结构体封装实现对区间结构的统一操作
同类题目: Sequence II, HDU 5147
          GTY's gay friends, HDU 5172
*/

```

例4-20 【线段树；区间修改；懒标记传递】快速矩阵操作（Fast Matrix Operations, UVa 11992）

有一个 r ($r \leq 20$) 行 c 列的全0矩阵，给出 m ($1 \leq m \leq 20\ 000$) 个操作，操作分3种，如表4-2所示。

表4-2 可执行的3种操作

操 作	备 注
1 $x_1\ y_1\ x_2\ y_2\ v$	子矩阵 (x_1, y_1, x_2, y_2) 的所有元素增加 v ($v > 0$)
2 $x_1\ y_1\ x_2\ y_2\ v$	子矩阵 (x_1, y_1, x_2, y_2) 的所有元素设为 v ($v > 0$)
3 $x_1\ y_1\ x_2\ y_2$	查询子矩阵 (x_1, y_1, x_2, y_2) 的元素和、最小值和最大值

子矩阵 (x_1, y_1, x_2, y_2) 是指满足 $x_1 \leq x \leq x_2$, $y_1 \leq y \leq y_2$ 的所有元素 (x, y) 。输入应保证任意时刻矩阵的所有元素之和都不超过 10^9 。

操作3中，将输出3个整数，即该子矩阵的元素和、最小值和最大值。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
const int MAXC = 1e6 + 4, INF = 1e9;
struct NodeInfo {
    int minv, maxv, sumv;
};
NodeInfo operator+(const NodeInfo &n1, const NodeInfo &n2) {
    return {min(n1.minv, n2.minv), max(n1.maxv, n2.maxv), n1.sumv + n2.sumv};
}

struct IntervalTree {
    NodeInfo nodes[MAXC];
    int setv[MAXC], addv[MAXC], qL, qR;
    bitset<MAXC> isSet;
    inline void setFlag(int o, int v) { setv[o] = v, isSet.set(o), addv[o] = 0; }
    void init(int n) {
        int sz = n * 2 + 2;
        fill_n(addv, sz, 0);
        isSet.reset(), isSet.set(1);
        memset(nodes, 0, sizeof(NodeInfo) * sz);
    }

    inline void maintain(int o, int L, int R) { // 维护信息
        int lc = o * 2, rc = o * 2 + 1, a = addv[o], s = setv[o];
        NodeInfo &nd = nodes[o], &li = nodes[lc], &ri = nodes[rc];
        if (R > L) nd = li + ri;
        if (isSet[o]) nd = {s, s, s * (R - L + 1)};
        if (a) nd.minv += a, nd.maxv += a, nd.sumv += a * (R - L + 1);
    }

    inline void pushdown(int o) { // 标记传递
        int lc = o * 2, rc = o * 2 + 1;
        if (isSet[o])
            setFlag(lc, setv[o]), setFlag(rc, setv[o]), isSet.reset(o); // 清除标记
    }
};

```

```

    if (addv[o])
        addv[lc] += addv[o], addv[rc] += addv[o], addv[o] = 0;    // 清除标记
}

void update(int o, int L, int R, int op, int v) {    // op(1:add, 2:set)
    int lc = o * 2, rc = o * 2 + 1, M = L + (R - L) / 2;
    if (qL <= L && qR >= R) {                    // 标记修改
        if (op == 1) addv[o] += v;                // add
        else setFlag(o, v);                        // set
    } else {
        pushdown(o);
        if (qL <= M) update(lc, L, M, op, v);
        else maintain(lc, L, M);
        if (qR > M) update(rc, M + 1, R, op, v);
        else maintain(rc, M + 1, R);
    }
    maintain(o, L, R);
}

NodeInfo query(int o, int L, int R) {
    int lc = o * 2, rc = o * 2 + 1, M = L + (R - L) / 2;
    maintain(o, L, R);
    if (qL <= L && qR >= R) return nodes[o];

    pushdown(o);
    NodeInfo li = {INF, -INF, 0}, ri = {INF, -INF, 0};
    if (qL <= M) li = query(lc, L, M);
    else maintain(lc, L, M);
    if (qR > M) ri = query(rc, M + 1, R);
    else maintain(rc, M + 1, R);
    return li + ri;
}

};

const int maxr = 20 + 5;
IntervalTree tree[maxr];

int main() {
    for (int r, c, m; scanf("%d%d%d", &r, &c, &m) == 3;) {
        for (int x = 1; x <= r; x++) tree[x].init(c);
        for (int i = 0, op, x1, y1, x2, y2, v; i < m; i++) {
            scanf("%d%d%d%d%d", &op, &x1, &y1, &x2, &y2);
            if (op < 3) {
                scanf("%d", &v);
                for (int x = x1; x <= x2; x++) {
                    IntervalTree &tx = tree[x];
                    tx.qL = y1, tx.qR = y2, tx.update(1, 1, c, op, v);
                }
            } else {
                NodeInfo gi = {INF, -INF, 0};

```

```

        for (int x = x1; x <= x2; x++) {
            IntervalTree &tx = tree[x];
            tx.qL = y1, tx.qR = y2, gi = gi + tx.query(1, 1, c);
        }
        printf("%d %d %d\n", gi.sumv, gi.minv, gi.maxv);
    }
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.2.4 节例题 10
注意：本题中如何通过 NodeInfo 封装区间的 3 个属性，以及如何通过重载 “+” 运算符实现对三者的统一操作
同类题目：Sequence Operation, HDU 3397
*/

```

例 4-21 【线段树；区间修改；乘法操作】维护序列（AHOI 2009，牛客NC 19889）

老师交给小可可一个维护数列的任务，现在小可可希望你来帮他完成。

有一个长为 n ($n \leq 10^5$) 的数列，不妨设为 $a_1, a_2, \dots, a_n$ 。有如下 3 种操作形式：

- ☐ 把数列中的一段数全部乘以一个值。
- ☐ 把数列中的一段数全部加上一个值。
- ☐ 询问数列中一段数的和，由于答案可能很大，你只需输出这个数模 P ($1 \leq P \leq 10^9$) 的值。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
int MOD; // 以下是各种模运算操作
LL _addM(LL a, LL b) { return (a + b) % MOD; }
LL _mulM(LL a, LL b) { return (a * b) % MOD; }
void addM(LL &a, LL b) { a = _addM(a, b); }
void mulM(LL &a, LL b) { a = _mulM(a, b); }

template <typename T, int SZ = 1000000>
struct IntervalTree {
    T addv[SZ], mulv[SZ], sum[SZ];
    int qL, qR;

    T build(int o, int l, int r, T *A) {
        mulv[o] = 1, addv[o] = 0;
        if (l == r) return sum[o] = A[l];
        int m = l + (r - l) / 2;
        return sum[o] = _addM(build(2 * o, l, m, A), build(2 * o + 1, m + 1, r, A));
    }
}

```

```

T query(int o, int L, int R) {
    if (qL <= L && R <= qR) return sum[o];
    int m = L + (R - L) / 2;
    T ans = 0;
    pushdown(o, L, R);
    if (qL <= m) addM(ans, query(2 * o, L, m));
    if (qR > m) addM(ans, query(2 * o + 1, m + 1, R));
    return ans;
}

void pushdown(int o, int L, int R) {
    int lc = 2 * o, rc = lc + 1, M = L + (R - L) / 2;
    LL &mo = mulv[o], &ao = addv[o];
    assert(L < R);
    if (mo != 1) {
        mulM(addv[lc], mo), mulM(addv[rc], mo);
        mulM(mulv[lc], mo), mulM(mulv[rc], mo);
        mulM(sum[lc], mo), mulM(sum[rc], mo);
        mo = 1;
    }
    if (ao != 0) {
        addM(addv[lc], ao), addM(addv[rc], ao);
        addM(sum[lc], _mulM(M - L + 1, ao)), addM(sum[rc], _mulM(R - M, ao));
        ao = 0;
    }
}

void update(int o, int L, int R, int op, T v) { // op = 1: add, 2: mul
    if (qL <= L && R <= qR) {
        if (op == 2) // ai += v
            addM(addv[o], v), addM(sum[o], _mulM(R - L + 1, v));
        else // ai *= v
            mulM(addv[o], v), mulM(sum[o], v), mulM(mulv[o], v);
        return;
    }

    int m = L + (R - L) / 2, lc = 2 * o, rc = lc + 1;
    pushdown(o, L, R);
    if (qL <= m) update(2 * o, L, m, op, v);
    if (qR > m) update(2 * o + 1, m + 1, R, op, v);
    sum[o] = _addM(sum[lc], sum[rc]);
}

};

const int NN = 1e5 + 8;
IntervalTree<LL, NN * 3> tree;
LL A[NN];

int main() {

```

```

ios::sync_with_stdio(false), cin.tie(0);
int N, M;
cin >> N >> MOD;
for (int i = 1; i <= N; i++) cin >> A[i], A[i] %= MOD;
tree.build(1, 1, N, A);
cin >> M;
for (int i = 0, o, v; i < M; i++) {
    cin >> o >> tree.qL >> tree.qR;
    if (o != 3)
        cin >> v, tree.update(1, 1, N, o, v);
    else
        cout << tree.query(1, 1, N) << endl;
}

return 0;
}
/*
注意：本题通过封装取模运算操作来简化代码
同类题目：An easy problem, HDU 5475
*/

```

例4-22 【线段树；区间修改；根号操作】花神游历各国（牛客NC50456）

花神喜欢步行游历各国，顺便虐爆各地竞赛。花神有一条游览路线，它是线型的。也就是说，所有游历国家呈一条线的形状排列。花神对每个国家都有一个喜欢程度（当然花神并不一定喜欢所有国家）。

每次旅行中，花神会选择一条游览路线，它在那一串国家中是连续的一段，这次旅行带来的开心值是这些国家的喜欢度的总和。当然，花神对这些国家的喜欢程序并不是恒定的，有时会突然对某些国家产生反感，使他对这些国家的喜欢度 δ 变为 $\sqrt{\delta}$ 。

给出花神每次的游览路线，以及开心度的变化，求出花神每次游览的开心值。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;

// 数组大小以及数据类型, 抽象成泛型
template <typename T = int, int SZ = 10004>
struct SegTree {
    struct Node { T max, sum; };
    Node NS[SZ * 3];
    int qL, qR;
    void init(int o, int L, int R, const T* A) { // 初始化操作
        assert(o < 3 * SZ);
        Node& nd = NS[o];
        if (L == R) { // 叶子区间
            nd.max = nd.sum = A[L];
        }
    }
};

```



```

        return;
    }
    int M = L + (R - L) / 2, lc = 2 * o, rc = 2 * o + 1;
    init(lc, L, M, A), init(rc, M + 1, R, A); // 左右区间分别初始化
    nd.max = max(NS[lc].max, NS[rc].max), nd.sum = NS[lc].sum + NS[rc].sum;
}

T querySum(int o, int L, int R) { // 查询区间和
    const Node& nd = NS[o];
    if (qL <= L && qR >= R) return nd.sum;
    int M = (L + R) / 2, lc = 2 * o, rc = 2 * o + 1;
    T ans = 0;
    if (qL <= M) ans += querySum(lc, L, M); // 左子区间
    if (qR > M) ans += querySum(rc, M + 1, R); // 右子区间
    return ans;
}

void make_sqrt(int o, int L, int R) {
    Node& nd = NS[o];
    if (nd.max <= 1) return; // sqrt 多次之后会变成 1, 之后不用再更新
    if (L == R) { // 叶子
        nd.max = floor(sqrt(nd.max)), nd.sum = floor(sqrt(nd.sum));
        return;
    }
    int M = (L + R) / 2, lc = 2 * o, rc = 2 * o + 1;
    if (qL <= M) make_sqrt(2 * o, L, M); // 覆盖左半边
    if (qR > M) make_sqrt(2 * o + 1, M + 1, R); // 覆盖右半边
    // 左右都更新, max 和 sum 也要更新
    nd.max = max(NS[lc].max, NS[rc].max), nd.sum = NS[lc].sum + NS[rc].sum;
}

};

const int NN = 1e5 + 4;
SegTree<LL, NN> st;
int N, M;
LL A[NN];
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> N;
    for (int i = 1; i <= N; i++) cin >> A[i];
    st.init(1, 1, N, A);
    cin >> M;
    for (int i = 1, x, l, r; i <= M; i++) {
        cin >> x >> st.qL >> st.qR;
        if (x == 1)
            cout << st.querySum(1, 1, N) << endl;
        else
            st.make_sqrt(1, 1, N);
    }
}

```

```

    return 0;
}
/*
注意：一个区间的元素在全部变成 1 之后，再进行开方操作就无任何效果
同类题目：Can you answer these queries?, HDU 4027
*/

```

例4-23 【线段树；区间修改；动态开点】山脉 (Mountains, IOI 2005, SPOJ NKMOU)

游乐园装了一种新的过山车，由 n ($1 \leq n \leq 10^9$) 条首尾相连的轨道组成 (见图4-9)，轨道编号为 $1 \sim n$ ，起点高度是0。 d_i 表示车辆经过轨道 i 之后的高度变化值， $d_i < 0$ 则表示高度下降，初始所有 $d_i = 0$ 。图4-9中的3个过山车， d_i 值分别是 $\{0, 0, 0, 0\}$ ， $\{2, 2, 2, 2\}$ ， $\{2, -1, 2, 2\}$ 。管理员有时候会重新配置区间 $[a, b]$ 内的所有轨道 i 来改变 d_i 。

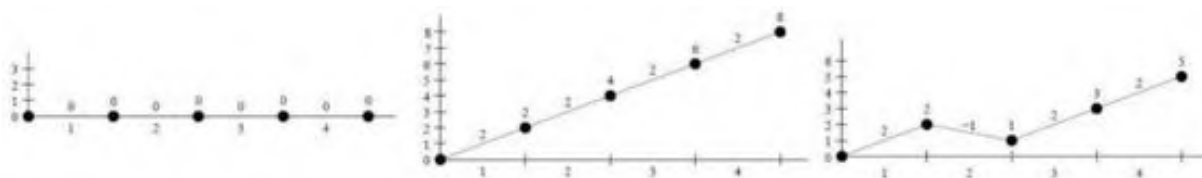


图4-9 过山车

玩过山车时，车辆会带着足够到达高度 h 的能量出发。也就是说，只要没到终点或者高度不超过 h ，车辆就会一直向前走。

给出如下形式的一些输入。

- ☐ I a b D : 表示重新配置 $[a, b]$ 中的所有轨道 i ，设置 $d_i = D$ ($-10^9 \leq D \leq 10^9$)。
- ☐ Q h : 表示有一趟车带着足够到达高度 h ($0 \leq h \leq 10^9$) 的能量出发，计算这趟车最多能走多远。
- ☐ E: 表示输入结束。

对于每一个形如 $Q\ h$ 的输入，输出上文所求的结果。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
struct Node {
    LL sum, maxp;                // 区间和，最大非负前缀
    Node *left, *right;
    int val;
    inline bool isleaf() { return !left && !right; }
    inline void init() { memset(this, 0, sizeof(Node)); }
    inline void delchildren() {
        if (left) delete left;
        if (right) delete right;
        left = right = nullptr;
    }
}
```

```

    ~Node() { delchildren(); } // C++析构函数，delete时会调用
};
typedef Node* PN;
int N;
PN root;
void maintain(Node& p) {
    p.sum = p.left->sum + p.right->sum;
    p.maxp = max(p.left->maxp, p.left->sum + p.right->maxp);
}
void setval(Node& p, int v, int L, int R) {
    assert(L <= R); // 整个区间设置值
    p.sum = (LL) (p.val = v) * (R - L + 1);
    p.maxp = max(0LL, p.sum);
    p.delchildren(); // 左右孩子都可以不要了
}
void pushdown(Node& p, int L, int R) {
    int M = (L + R) / 2;
    p.left = new Node(), p.right = new Node();
    setval(*p.left, p.val, L, M), setval(*p.right, p.val, M + 1, R);
}
void modify(int l, int r, int v, Node& p = *root, int nL = 1, int nR = N) {
    int M = (nL + nR) / 2;
    if (l <= nL && nR <= r) {
        setval(p, v, nL, nR);
        return;
    }
    if (p.isleaf()) pushdown(p, nL, nR); // 左右区间创建子结点
    if (l <= M) modify(l, r, v, *p.left, nL, M); // 递归
    if (r > M) modify(l, r, v, *p.right, M + 1, nR); // 递归
    maintain(p); // 维护 sum 以及 maxp
}
// 查询数组中前缀和小于等于 h 的最长前缀的右端点位置
int query(LL h, Node& p = *root, int L = 1, int R = N) {
    if (h >= p.maxp) return R; // 整个区间都行
    if (p.isleaf()) return L + (h / p.val) - 1; // 区间元素都相等，按比例返回
    int M = (L + R) / 2;
    Node& pl = *p.left;
    return h >= pl.maxp
        ? query(h - pl.sum, *p.right, M + 1, R) // 左边最大非负前缀小于等于 h，一定能跑到右边
        : query(h, pl, L, M); // 只能在左子区间内部
}

void dbgprint(Node& p = *root, int L = 1, int R = N) { // 打印数组，调试用
    if (p.isleaf()) {
        for (int i = L; i <= R; i++) printf("%d ", p.val);
        return;
    }
    int M = (L + R) / 2;

```